23CSE455 Design Patterns
Part 1 Findings: Pattern Discovery

Design Pattern Identification and Evolution in a Spring MVC-Based Warehouse Management Application

Team: B9
Date: 11 October 2026 (re-audited findings)
Team working repository: akdswordguy/Team_B9_DP
Companion documents: `B7_Report.tex` (full report), `report/part2_findings.tex`, `report/part3_findings.tex`

Roll number	Student name	Batch
AM.SC.U4CSE23215	D Aditya Kiran	CSE C
AM.SC.U4CSE23235	Mithun Krishna	CSE C
AM.SC.U4CSE23248	Sai Kishen K M	CSE C
AM.SC.U4CSE23359	Sivadev Smijith	CSE D

Evidence paths in this document are relative to `Team_B7_Findings/` in the team working repository unless stated otherwise. OpenWMS source paths are relative to `org.openwms.core.util/src/main/java/org/openwms/core/` in the upstream repository.

1 Scope and baseline

Item	Value
Application	OpenWMS.org, an open-source warehouse management system
Repository	`openwms/org.openwms` (CORE)
Baseline commit	`9d263bf65ec75301e0302539ae7bf581412742b8`
Commit message and date	`#185 development version 4.1.0-SNAPSHOT`, 13 July 2026
Module inspected	`org.openwms.core.util` (utilities, exception types, shared event types)
Java files at baseline	64
Licence	Apache License 2.0

Architecture context. OpenWMS uses a microservice architecture: each business component (Common, Inventory, Picking, Receiving, Shipping, and others) is a Spring Boot service in its own repository. The CORE README describes RabbitMQ with Spring Integration as the event broker for asynchronous events and commands. The CORE repository itself is a library, not a service, so Part 1 makes no source-level claim about services whose code is not in it. The README lists the Picking Library as a private GPLv3 repository and the Inventory Service as private preview, which is why Part 2 uses reference implementations for those changes.

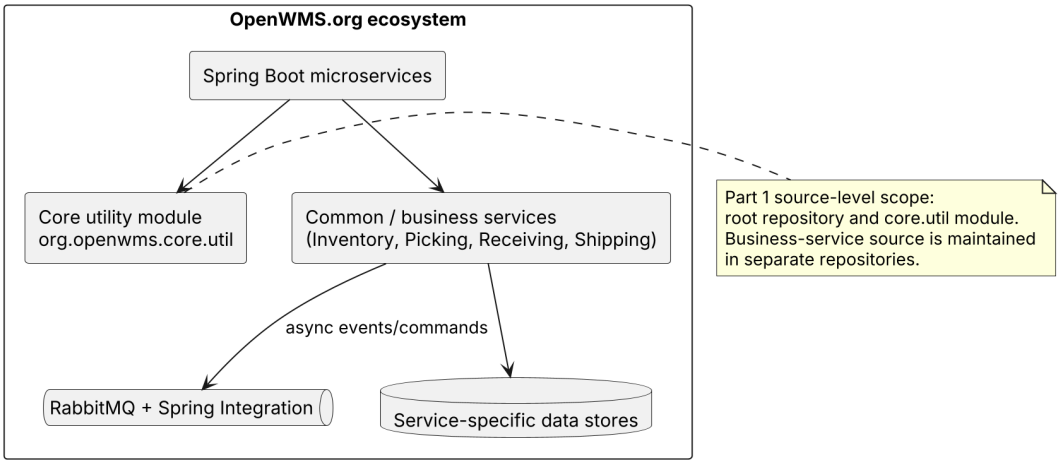


Figure 1. Architecture scope (`Part1_Findings/diagrams/architecture_scope.puml`).

2 Method and verification

Candidates were identified from the Java source at the baseline commit. A candidate was confirmed only when the source demonstrated the relationships and behavior the pattern requires; class names, Spring annotations, framework-generated proxies, or an LLM assertion alone were not accepted. Each candidate was classified as a GoF pattern, an architectural/framework mechanism, or rejected.

Re-audit (11 October 2026). The upstream repository was cloned afresh and checked out at the baseline. Every line range cited in `Part1_Findings/Part1_Findings.md` and `Part1_Findings/data/patterns.csv` was compared with the source and found exact. The ten files under `Part1_Findings/evidence/source/` are byte-identical to upstream. The 64-file count was confirmed. The corrections made are listed in Section 8.

3 Summary of results

Measure	Count	Detail
Confirmed GoF pattern types	2	Observer, Composite
Confirmed GoF pattern instances	2	P01 Core event subscription and dispatch; P02 `TreeNode` composite tree
Concrete participant classes	2	`SimpleEventDispatcher`, `TreeNodeImpl<T>`
Unique participant symbols	7	`EventBroker`, `EventDispatcher`, `SimpleEventDispatcher`, `EventListener`, `RootApplicationEvent`, `TreeNode`, `TreeNodeImpl`
Non-GoF mechanisms recorded	3	N01 async publish/subscribe; N02 Spring listener idiom; N03 AOP advice (partially confirmed)
Rejected GoF candidates	2	Singleton, Proxy
Uncertain candidates	0	None recorded in the frozen inventory

These are repository-scope counts. They do not claim that these are the only patterns in the whole OpenWMS product.

4 P01 — Observer: core event subscription and dispatch

Category. GoF behavioral pattern. **Origin.** OpenWMS source, not introduced by this case study.

Purpose. Decouple code that raises application events from the listeners that react to them, through explicit subscription, unsubscription, and dispatch.

4.1 Participant mapping

GoF role	Symbol	Source evidence
Subject: attach/detach	`EventBroker` (interface)	`event/EventBroker.java:19-56`. Two `subscribe` and two `unsubscribe` methods, keyed by event class, taking a listener instance or a Spring bean name.
Subject: notify	`EventDispatcher` (interface)	`event/EventDispatcher.java:19-34`. Extends `EventBroker`; adds `<TextendsRootApplicationEvent>voiddispatch(Tevent)`. Its Javadoc leaves sync/async delivery undefined.
ConcreteSubject	`SimpleEventDispatcher` (class)	`event/SimpleEventDispatcher.java:27-38` (in-memory `Map<Class,Set<EventListener>>`, documented as sequential and synchronous); `:54-56` subscribe; `:75-79` unsubscribe; `:99-106` dispatch.
Observer	`EventListener` (interface)	`event/EventListener.java:19-31`. Single update method `onEvent(RootApplicationEvent)`.
Notification payload	`RootApplicationEvent` (class)	`event/RootApplicationEvent.java:23-29`. Base event type; extends Spring `ApplicationEvent`.
ConcreteObserver	—	None: no class in the 64 Java files implements `EventListener`.

4.2 Key behavior

The notification step is the core of the pattern. In `SimpleEventDispatcher` (excerpt from OpenWMS, Apache License 2.0):

```
99  public <T extends RootApplicationEvent> void dispatch(T event) {
100     if (event == null) {
101         return;
102     }
103     var listeners = subscriptions.get(event.getClass());
104     if (listeners != null) {
105         listeners.forEach(l -> l.onEvent(event));
106     }
```

The subject iterates its registered observers and calls the common update method, without knowing any concrete listener type. Registration (`subscribe`) adds to a concurrent set per event class; `unsubscribe` removes from it.

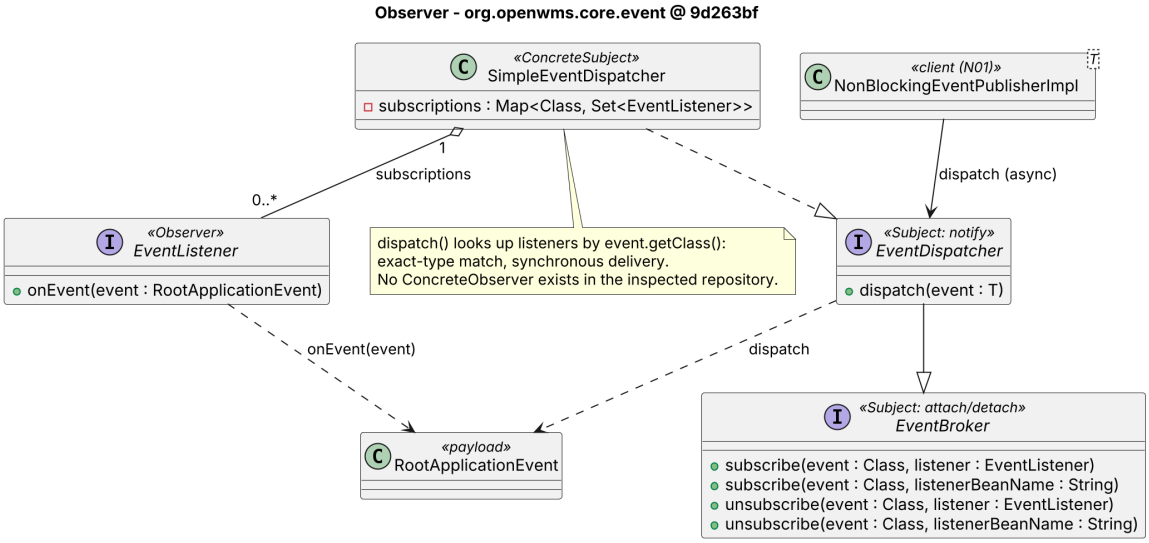


Figure 2. Observer roles (`Part1_Findings/diagrams/observer_uml.puml`).

4.3 Verification, observations and limitations

- **Confirmed from source.** Runtime attach/detach and notification through a common interface are both present. The GoF Subject role is distributed across `EventBroker` (attach/detach), `EventDispatcher` (notify) and `SimpleEventDispatcher` (ConcreteSubject); no extra `Subject` type is invented.
- **Exact-type matching.** Line 103 looks up listeners by `event.getClass()`. A listener subscribed to `RootApplicationEvent` is not notified of a subclass event such as `UserChangedEvent`.
- **Synchronous delivery.** Observers are called sequentially in the dispatching thread. Asynchrony is added only by the separate publisher (N01).
- **No ConcreteObserver in scope.** The repository supplies the infrastructure; concrete listeners are expected in the service repositories. The instance is confirmed at library level.
- **Publisher is a client.** `NonBlockingEventPublisherImpl` calls `dispatch`; in GoF terms that is a client that triggers notification, so it is recorded as N01 and not counted as a participant.

5 P02 — Composite: `TreeNode` composite tree

Category. GoF structural pattern (transparent variant, single implementation class). **Origin.** OpenWMS source.

Purpose. Represent a recursive tree in which every node stores its children through the same `TreeNode` abstraction, and leaves are distinguished from containers by whether the child collection is empty.

5.1 Participant mapping

GoF role	Symbol	Source evidence
Component	`TreeNode<T>` (interface)	`util/TreeNode.java:23-28` abstraction; `:45-49` `isLeaf`; `:51-72` `getChildren`; `:75-115` `getChild`, `addChild`, `removeChild`, `getParent`, `setParent`. Children are typed `TreeNode<T>`.
Leaf and Composite	`TreeNodeImpl<T>` (class)	`util/TreeNodeImpl.java:29-33` `parent` and `LinkedHashMap childrenMap`; `:47-57` `addChild` sets `child.setParent(this)`; `:64-68` `removeChild` clears the parent; `:83-108` parent access, `getChildren`, `isLeaf()` = `childrenMap.isEmpty()`.
Client	—	None: no class in the repository uses `TreeNode` or `TreeNodeImpl`.

```
54 @Override
55 public void addChild(Object identifier, TreeNode<T> child) {
56     child.setParent(this);
57     childrenMap.put(identifier, child);
```

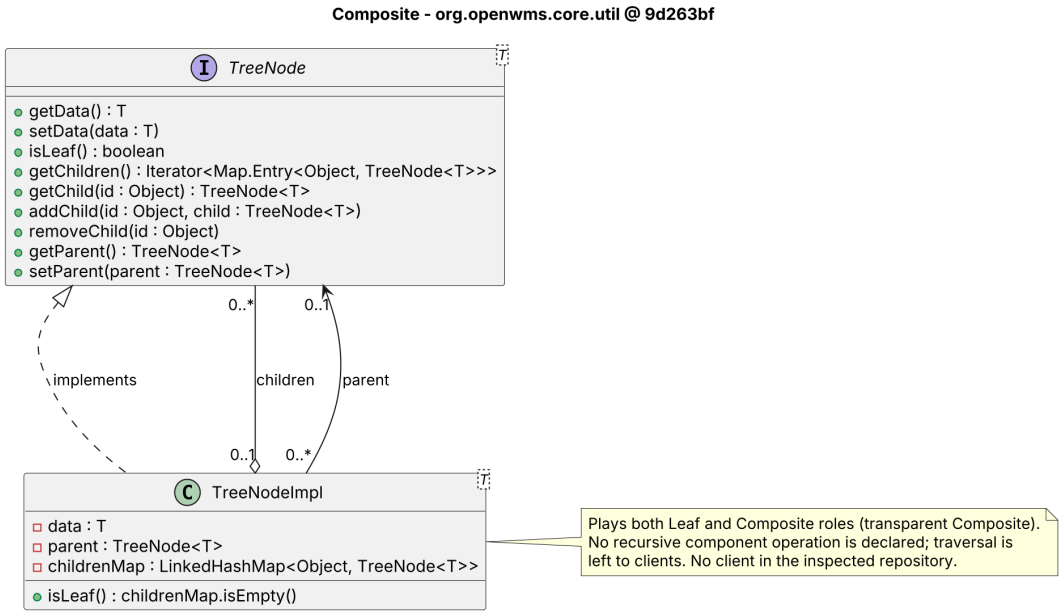


Figure 3. Composite roles (`Part1_Findings/diagrams/composite_uml.puml`).

5.2 Verification, observations and limitations

- **Confirmed from source.** Part–whole recursion (children are themselves `TreeNode<T>`) and uniform treatment (child management declared on the component interface) are both present.
- **Transparent, single-class variant.** Declaring child management on the Component is the GoF transparent form. There are no separate Leaf and Composite classes; "leaf" is a runtime state.
- **No recursive component operation.** GoF Composite normally has an operation that a composite forwards to its children. `TreeNode` declares only data access, navigation and child management, and leaves traversal to the client. The classification therefore rests on recursive structure and uniform interface, not recursive behavior.

- **No client in scope.** This is a confirmed utility-level instance, not evidence that a warehouse workflow uses Composite.

6 Non-GoF mechanisms (kept out of the GoF totals)

ID	Mechanism	Evidence and reasoning	Status
N01	Asynchronous publish/subscribe	`event/EventPublisher.java:19-33` declares `publish(T)`. `event/NonBlockingEventPublisherImpl.java:22-50` receives an `EventDispatcher` by constructor and its `@Asyncpublish` calls `dispatcher.dispatch(event)`. An eventing mechanism layered on P01, not a second Observer instance.	Confirmed
N02	Spring application-event listener	`startup/LocalServiceInitializer.java:27-33` implements `ApplicationListener<ContextRefreshedEvent>`; `:50-53` publishes a `MergePropertiesEvent` through the `ApplicationContext`. The subscription infrastructure is Spring's, so it is a framework idiom.	Confirmed
N03	Fire-after-transaction AOP advice	`aop/FireAfterTransactionAspect.java:30-36` documents firing events after a transaction; `:76-86` and `:95-107` publish configured events, the second with `@Async`. However, the class is a plain `@Component`: no `@Aspect`, pointcut, advice annotation, XML `aop:` configuration or AspectJ dependency exists in the repository. The advice methods exist; the interception wiring must live in a consuming service.	Partially confirmed

7 Rejected candidates

Candidate	Reason for rejection
Singleton	Spring `@Component` scope alone does not establish GoF Singleton. The candidate beans (`SimpleEventDispatcher`, `NonBlockingEventPublisherImpl`, `LocalServiceInitializer`, `FireAfterTransactionAspect`) all have public constructors and no static instance or accessor; they are single only because the container scopes them that way.
Proxy	Framework-generated proxies and AOP behavior are not counted as an application-level GoF Proxy, and no application class controls access to a separate subject through the same interface.

These are scope-bounded rejections, preserved in `Part1_Findings/data/rejected_candidates.csv`; they do not prove the patterns are absent from the wider ecosystem.

8 Corrections made in the re-audit

Item	Previously	Now
N03 AOP mechanism	Recorded as a confirmed AOP mechanism.	Partially confirmed: advice methods exist, but no aspect or pointcut wiring is in the repository.
Observer limitations	Not recorded.	Exact-type matching, synchronous delivery, and absence of a ConcreteObserver recorded.
Composite limitations	Only the missing client was recorded.	Transparent single-class variant and absence of a recursive component operation recorded.
Singleton rejection	General statement about Spring scope.	Reason now names the candidate beans and the missing static instance/accessor.
`composite_uml.puml`	Placed `parent`/`children` on the interface with unclear multiplicity.	Members and multiplicities match the source; limitations noted on the diagram.
`observer_uml.puml`	Role names and the publisher client were not shown.	GoF role stereotypes, the N01 client and the exact-type note added.
`main.tex` (Part 1)	Evidence column said only "see findings"; diagram text claimed recursive traversal.	Exact file:line evidence; traversal claim removed; N01–N03 summarised.

The totals (2 types, 2 instances, 2 concrete classes, 7 symbols, 2 rejected) are unchanged by the corrections.

9 Evidence files

- `Part1_Findings/Part1_Findings.md` — full findings with line citations.
- `Part1_Findings/data/patterns.csv`, `pattern_counts.csv`, `rejected_candidates.csv` — machine-readable inventory.
- `Part1_Findings/evidence/verification_notes.md` — human verification notes and re-audit record.
- `Part1_Findings/evidence/source/org.openwms/` — byte-identical upstream excerpts with the Apache 2.0 `LICENSE`.
- `Part1_Findings/diagrams/` — PlantUML sources and rendered PNGs.
- `LLM_RECORDS/LLM_RECORDS.md` — records P1-01 and AUDIT-01.